
1 Evaluación 4 - Sumativa: Programa básico en Python

NOMBRE: Leonel Cartes, Benjamin Morales y Felipe Echeverria
ASIGNATURA: introducción a la programación y robótica aplicada
SECCION: ETDI01
PROFESOR: Jacqueline Lisette San Martin Grandon
FECHA: 25-05-2026

Contenido

2	Introducción	2
3	Desarrollo del código en Python a partir del diagrama de flujo:	3
4	. Relación entre el diagrama y el código	3
4.1.1	. Sentencias aritméticas y lógicas	3
5	. Secuencias	4
6	. Funciones	4
7	. Requerimientos de programación	5
8	. Uso del lenguaje	5
9	. Funcionalidad y estructura	6
10	. Aplicación del código	6
11	. Ejemplo de ejecución:	6
12	. Anexo del código	7
13	. Conclusión	11

2 Introducción

En este informe como grupo tenemos como objetivo interpretar y aplicar algoritmos en lenguaje Python mediante el desarrollo de un programa que simula el registro de usuarios. El código implementamos las

etapas descritas en el diagrama de flujo (DFD), validando cada ingreso de datos y asegurando que el flujo de las funciones se cumpla correctamente. El desarrollo de esta evaluación nos permitió comprender cómo los diagramas de flujo se logran traducir en instrucciones de programación reales, fortaleciendo y la capacidad de resolver problemas en el ámbito del desarrollo profesional.

3 Desarrollo del código en Python a partir del diagrama de flujo:

4 . Relación entre el diagrama y el código

El programa implementa de manera directa la lógica representada en el diagrama de flujo, donde cada bloque se traduce en una función o estructura de control en Python:

- `validar_rut()` → corresponde al bloque “*Ingrese RUT*” y su validación (`Val = existe_RUT(RUT)`).
- `validar_contraseña()` → refleja el bloque “*Crear contraseña*” y la verificación de validez (`contraseña_valida(Contraseña)`).
- `registrar_datos()` → equivale al bloque “*RegistrarDatos()*”, mostrando la información ingresada.
- `datos_repetidos()` → asegura la integridad del sistema, evitando duplicación de RUT y contraseña.
- Ciclo `while pregunta.lower()=="si":` → reproduce la decisión “*¿Desea continuar ingresando?*”, permitiendo repetir el proceso hasta que el usuario indique lo contrario.

De esta forma, el código se mantiene fiel al diagrama de flujo, garantizando que cada etapa del algoritmo esté representada en una función o estructura de control específica.

4.1.1 . Sentencias aritméticas y lógicas

El programa utiliza sentencias lógicas y relacionales para validar condiciones en cada etapa del registro de usuarios. Estas sentencias son fundamentales para controlar el flujo del algoritmo y garantizar la integridad de los datos:

Formato del rut:

```
if len(rut_pri) == 8 and len(rut_sec) == 1:
```

Verifica que el RUT tenga exactamente 8 dígitos más un dígito verificador, aplicando el operador lógico `and`.

Longitud de contraseña:

```
if len(contraseña) > 0 and len(contraseña)
```

Controla que la contraseña no esté vacía y no supere los 20 caracteres, combinando operadores relacionales (`>`, `<=`) con el lógico `and`.

Verificación de duplicado:

```
if not repetidos:
```

Evalúa si los datos ingresados (RUT y contraseña) son únicos antes de crear la cuenta, utilizando el operador lógico `not`.

5 . Secuencias

El programa sigue una secuencia lógica de ejecución:

1. Solicitud de datos personales.
2. Validación de cada campo.
3. Verificación de duplicados.
4. Registro y visualización de datos.
5. Repetición o finalización del proceso.

Cada paso depende del anterior, lo que refleja una estructura secuencial y condicional típica de los diagramas de flujo.

6 . Funciones

El código está dividido en funciones que encapsulan tareas específicas:

- `validar_rut()`
 - ✓ Propósito: *Garantizar que el RUT ingresado cumpla con el formato correcto (xxxxxxx-x), donde los primeros 8 caracteres son numéricos y el último es el dígito verificador.*

Aspectos técnicos:

- ✓ Usa un bucle *while True* para repetir la solicitud hasta que se cumpla la condición.
- ✓ Emplea *operadores lógicos (and)* y *función len()* para validar longitud.
- ✓ Retorna el valor concatenado con el guion (*rut_pri + "-" + rut_sec*), lo que demuestra el uso de operaciones de concatenación de cadenas.
- ✓ La estructura condicional *if/else* controla el flujo de validación y retroalimentación al usuario.

- `validar_contraseña()`
 - ✓ Propósito: *Limitar la longitud de la contraseña a un máximo de 20 caracteres y evitar entradas vacías.*

Aspectos técnicos:

- ✓ También usa un *bucle infinito (while True)* para garantizar la validación continua.
- ✓ Aplica *operadores relacionales (>, <=)* para verificar el rango permitido.
- ✓ Retorna la contraseña válida, lo que permite su reutilización en otras funciones.

- `registrar_datos()`
 - ✓ Propósito: *Almacenar los datos del usuario en un diccionario y mostrar la información si el usuario lo solicita.*

Aspectos técnicos:

- ✓ Crea un diccionario (*usuario = {...}*), estructura clave-valor que permite agrupar datos relacionados.
- ✓ Usa acceso por clave (*usuario["nombre"]*) para imprimir valores específicos.
- ✓ Implementa una decisión condicional basada en la respuesta del usuario (*if visor_datos.lower() == "si"*).
- ✓ No retorna valores, pero actúa como función de salida y visualización.

- `datos_repetidos()`

- ✓ Propósito: Evitar duplicación de datos en la lista global usuarios.

Aspectos técnicos:

- ✓ Recorre la lista usuarios con *un bucle for*, iterando sobre cada diccionario almacenado.
- ✓ Compara valores mediante *operadores de igualdad (==)*.
- ✓ *Retorna True* si encuentra coincidencias, lo que permite controlar el flujo en el programa principal.
- ✓ Usa estructura de control secuencial y condicional para verificar.

7 . Requerimientos de programación

El programa cumple con los requerimientos esenciales de un sistema de registro en Python, asegurando un flujo controlado y confiable:

- **Validación de datos:** cada ingreso es revisado antes de continuar, evitando campos vacíos o formatos incorrectos mediante condiciones lógicas y repetición de funciones hasta obtener entradas válidas.
- **Flujo correcto:** las funciones se ejecutan en el orden establecido y se repiten hasta que se cumpla la condición indicada, garantizando coherencia en el proceso de registro.
- **Control de errores:** se previene el ingreso inválido mediante mensajes de retroalimentación al usuario, lo que asegura integridad y consistencia en los datos almacenados.
- **Interacción con el usuario:** se solicita confirmación en cada etapa, permitiendo decidir si continuar con el registro, crear nuevas cuentas o finalizar el sistema.

8 . Uso del lenguaje

Python se emplea en este programa gracias a su sintaxis clara y estructura flexible, lo que facilita la implementación de algoritmos de validación y registro de datos. El código aprovecha:

- **Listas y diccionarios**

- ✓ *La lista usuarios=[]* permite almacenar múltiples registros de manera ordenada.

- ✓ *Los diccionarios (usuario={...}) agrupan información relacionada en pares clave-valor, garantizando acceso rápido y estructurado a los datos.*
- **Bucles y estructuras condicionales**
 - ✓ El uso de *while* asegura la repetición de procesos hasta cumplir condiciones específicas (validación de entradas o decisión del usuario).
 - ✓ Las sentencias *if/else* permiten controlar el flujo lógico, evaluando condiciones y ejecutando acciones según los resultados.
- **Funciones reutilizables**
 - ✓ La modularización mediante funciones (*validar_rut, validar_contraseña, registrar_datos, datos_repetidos*) mejora la legibilidad y facilita la reutilización del código.
 - ✓ Cada función encapsula una tarea específica, lo que refleja buenas prácticas de programación estructurada.

9 . Funcionalidad y estructura

El programa implementa un flujo iterativo y condicional que refleja fielmente la lógica del diagrama de flujo:

- **Iteración:** permite repetir la creación de cuentas tantas veces como el usuario lo desee.
- **Validación:** cada campo ingresado es verificado antes de avanzar, garantizando la integridad de los datos.
- **Condicionalidad:** el sistema finaliza cuando el usuario responde “no”, asegurando un cierre controlado del proceso.

Este comportamiento demuestra comprensión del ciclo de vida de un algoritmo, integrando decisiones, repeticiones y validaciones en un flujo coherente y estructurado.

10 . Aplicación del código

El sistema puede aplicarse en contextos reales como:

- **Educación:** registro de estudiantes y control básico de datos en plataformas académicas.
- **Empresas:** administración de empleados y validación de información en sistemas internos.
- **Formularios digitales:** verificación de datos personales en aplicaciones web o móviles.
- **Autenticación básica:** simulación de creación de cuentas e inicio de sesión en proyectos simples.
- **Aprendizaje de programación:** práctica de diagramas de flujo, sentencias lógicas y funciones en Python.

11 . Ejemplo de ejecución:

Bienvenido al sistema de registro de cuentas. ¿Desea continuar? (si/no): si

¿Desea crear su cuenta? (si/no): si

Ingrese su nombre: Juan



Ingrese su apellido paterno: Pérez

Ingrese su apellido materno: Gonzales

Ingrese los primeros 8 dígitos del RUT: 12345678

Ingrese el dígito verificador de su rut: 9

Ingrese la contraseña que desea utilizar (máximo 20 caracteres): clave123

Cuenta creada exitosamente.

¿Desea ver los datos ingresados? (si/no): si

Nombre: Juan

Apellido paterno: Perez

Apellido materno: Gonzales

RUT: 12345678-9

Contraseña: clave123

12 . Anexo del código

#Variable para almacenar los datos de los usuarios registrados y para evitar que se repitan los datos de otras cuentas ingresadas.

```
usuarios=[]
```

#Funcion para validar el formato del RUT permitiendo que solo se ingresen los 8 primeros caracteres y despues el digito verificador.

```
def validar_rut():
```

```
    while True:
```

```
        rut_pri=input("Ingrese los primeros 8 digitos del RUT: ")
```

```
        rut_sec=input("Ingrese el digito verificador de su rut: ")
```

```
        if len(rut_pri)==8 and len(rut_sec)==1:
```

```
            rut=rut_pri+"-"+rut_sec
```

```
            return rut
```

```
        else:
```

```
            print("Su RUT no es valido, el formato debe ser asi xxxxxxxx-x")
```

#Funcion para validar la contraseña y que solo se puedan ingresar 20 caracteres.

```
def validar_contraseña():
```

```
    while True:
```

```
        contraseña=input("Ingrese la contraseña que desea utilizar(maximo 20 caracteres): ")
```

```
        if len(contraseña)>0 and len(contraseña)<=20:
```

```
            return contraseña
```

```
        else:
```

```
            print("Contraseña invalida. La contraseña no debe exceder los 20 caracteres.")
```

#Funcion para que se registren los datos de un usuario.

```
def registrar_datos(nombre, apellido_pat, apellido_mat, rut, contraseña):
```

```
    usuario={"nombre": nombre,
```

```
            "apellido_pat": apellido_pat,
```

```
            "apellido_mat": apellido_mat,
```

```
            "rut": rut,
```

```
            "contraseña": contraseña}
```

```
visor_datos=input("¿Desea ver los datos ingresados? (si/no): ")
```

```
if visor_datos.lower()=="si":
```

```
    print("Nombre:", usuario["nombre"])
```

```
    print("Apellido paterno:", usuario["apellido_pat"])
```

```
    print("Apellido materno:", usuario["apellido_mat"])
```

```
    print("RUT:", usuario["rut"])
```

```
    print("Contraseña:", usuario["contraseña"])
```

```
else:
```

```
    print("No se mostrarán los datos ingresados.")
```




#Funcion para verificar que no se repitan datos al crear mas cuentas.

```
def datos_repetidos(rut, contraseña):
```

```
    for usuario in usuarios:
```

```
        if usuario["rut"]==rut:
```

```
            print("El RUT ingresado ya esta registrado pruebe con otro")
```

```
            return True
```

```
        if usuario["contraseña"]==contraseña:
```

```
            print("La contraseña ingresada ya esta registrada pruebe con otra")
```

```
            return True
```

```
    return False
```

#Codigo principal del programa.

```
pregunta=input("Bienvenido al sistema de registro de cuentas. ¿Desea continuar? (si/no): ")
```

```
while pregunta.lower()=="si":
```

```
    crear_cuenta=input("¿Desea crear su cuenta? (si/no): ")
```

```
    if crear_cuenta.lower()=="si":
```

```
        while True:
```

```
            nombre=input("Ingrese su nombre: ")
```

```
            if len(nombre)>0:
```

```
                break
```

```
            else:
```

```
                print("El nombre no puede estar vacio.")
```

```
        while True:
```

```
            apellido_pat=input("Ingrese su apellido paterno: ")
```

```
            if len(apellido_pat)>0:
```

```
                break
```

```
            else:
```

```
print("El apellido paterno no puede estar vacio.")
```

```
while True:
```

```
    apellido_mat=input("Ingrese su apellido materno: ")
```

```
    if len(apellido_mat)>0:
```

```
        break
```

```
    else:
```

```
        print("El apellido materno no puede estar vacio.")
```

```
rut=validar_rut()
```

```
print("RUT:", rut)
```

```
contraseña=validar_contraseña()
```

```
print("Contraseña:", contraseña)
```

```
repetidos=datos_repetidos(rut, contraseña)
```

```
if not repetidos:
```

```
    usuarios.append({"rut": rut,"contraseña": contraseña})
```

```
    print("Cuenta creada exitosamente.")
```

```
    registrar_datos(nombre,apellido_pat,apellido_mat,rut,contraseña)
```

```
else:
```

```
    print("No se creo la cuenta debido a datos repetidos.")
```

```
pregunta=input("¿Desea crear otra cuenta? (si/no): ")
```

```
else:
```

```
    print("No se creo una cuenta.")
```

```
    pregunta="no"
```

```
print("Gracias por utilizar el sistema de registro de cuentas. Hasta luego")
```

13 . Conclusión

En conclusión, el desarrollo de este programa permitió aplicar conceptos fundamentales de programación en Python, tales como funciones, estructuras condicionales, validación de datos y ciclos iterativos. Además, se logró interpretar correctamente un diagrama de flujo y transformarlo en una solución funcional. Como grupo, esta actividad fortaleció habilidades de trabajo colaborativo, lógica computacional y resolución de problemas, competencias esenciales para el desarrollo profesional en el área informática.